Lesson 1: Classes, and what a network learns

Two videos on neural networks, then a Vector2D class that makes + and * work on vectors.

Duration	About 1 h 40 min. If time runs out, stop after block 2.
File	`01_python_warmup/vectors.py`, empty at the start.
Before	Environment set up as in `README.md`. A `# %%` cell runs in the VS Code interactive window. The demo and both videos are open in browser tabs.
Goals	The learner writes a class with `__init__` and methods, can say what `self` is, makes `+` and `*` work on their own objects, and can explain why `2 * u` needs `__rmul__`.

How to run this session

- Ask for a prediction before every run. A wrong prediction followed by a surprise teaches more than a right one.
- Give hints in the order listed, and only after a real attempt. Never type on the learner's keyboard.
- When Python raises an error, the learner reads the last line aloud before anyone touches the code. Every message quoted here was checked on Python 3.14.
- Green boxes are for you. Do not read them out.
- If block 2 takes the whole time, stop there. Classes must be solid before micrograd.

Block	Time	Where
1. Videos and two questions	40 min	Browser
2. The `Vector2D` class	45 min	`vectors.py`
3. Wrap-up and commit	10 min	`README.md`, terminal

Block 1: videos and two questions

Start with the interactive digit demo, before any video. The learner draws a digit and watches the network guess it.

- Hook: the interactive digit demo on 3Blue1Brown
- Chapter 1, But what is a neural network?, about 19 min
- Chapter 2, Gradient descent, how neural networks learn, about 21 min

> **ASK**
> After both videos:
> 1. What does learning change inside the network?
> 2. Why do we need derivatives?
>
> Write the answers down word for word and do not correct them. Lesson 2 ends by asking the same questions again.

EXPECTED (FOR YOU)

1. The weights and biases, 13,002 numbers for the digit network in the video. The layers, the neurons and the activation function stay fixed.
2. The cost function scores how wrong the network is on the training examples. The derivative of the cost with respect to one weight says which way to nudge that weight to lower the cost, and how strongly the cost reacts. Gradient descent repeats small nudges downhill.

WATCH FOR

"The neurons change" or "the network grows new connections". Note the answer and move on. Lesson 2 revisits it with a concrete example.

Block 2: the `Vector2D` class

The class lives in the top cell of `vectors.py`. It grows at each step, and the learner re-runs that cell after every change. The checks for each step go in their own `# %%` cell below it. This guide calls the class `Vector2D`, but any name works.

WATCH FOR

`AttributeError: 'Vector2D' object has no attribute 'norm'` right after adding `norm`. The object `u` was created before the class cell was re-run, so it still belongs to the old class. Re-run the class cell, then create `u` again. This one comes back at every step.

Step 0: a hook, before any class (5 min)

ASK

How would you store the point (3, 4) in Python today? Then predict `(3, 4) + (1, 2)` and `[3, 4] * 2`.

EXPECTED (FOR YOU)

A tuple or a list. Python prints `(3, 4, 1, 2)` and `[3, 4, 3, 4]`. It glues sequences together and knows nothing about vectors.

Leave the next question open. The rest of the block answers it.

ASK

How could we make `+` mean vector addition?

Step 1: the class shell

TASK

Write a class so that `u = Vector2D(3, 4)` stores both coordinates and `print(u.x, u.y)` shows `3 4`. A second vector `v = Vector2D(1, 2)` prints its own values.

ASK

What is `self`? How does `u` know it is not `v`?

EXPECTED (FOR YOU)

`self` is the object being built or used. `Vector2D(3, 4)` creates an empty object, then calls `__init__` on it with `3` and `4`. Once `norm` exists, show that `u.norm()` and `Vector2D.norm(u)` are the same call.

IF STUCK, IN THIS ORDER

1. Show the `__init__` pattern on a different class, such as a `Dog` with a name and an age. Never show the `Vector2D` version. Carrying the pattern over is the exercise.
2. "Which method does Python call when you write `Vector2D(3, 4)`?"

WATCH FOR

- `Vector2D.__init__() takes 2 positional arguments but 3 were given` means `self` is missing from the parameters. Ask the learner to count what Python passed, the new object included.
- `Vector2D() takes no arguments` means Python found no `__init__`, usually because of a typo such as `__int__` or single underscores. It fell back on the default, which takes nothing.

Step 2: `norm()`

ASK

Predict `Vector2D(3, 4).norm()`.

EXPECTED (FOR YOU)

`5.0`. Follow up with: why `5.0` and not `5`? `math.sqrt` and `** 0.5` always return a float.

IF STUCK, IN THIS ORDER

1. "Draw the vector. Which theorem gives the length of the hypotenuse?"
2. Point to `math.sqrt`, or to `** 0.5`.

WATCH FOR

`print(u.norm)` without parentheses prints `<bound method Vector2D.norm of ...>`. The method exists, but nothing called it.

Step 3: `__repr__`

ASK

Before writing anything, predict what `print(u)` shows.

EXPECTED (FOR YOU)

Something like `<__main__.Vector2D object at 0x1024e06e0>`. Ask what the hex number might be. It is where the object lives in memory.

TASK

Write a `__repr__` method so that `print(u)` shows `Vector2D(3, 4)`. If the learner asks how, point to `__repr__` in the Python docs.

WATCH FOR

`TypeError: __repr__ returned non-string (type NoneType)` means `__repr__` calls `print` instead of `return`. The message states the problem, so have the learner read it aloud and explain it.

Optional question: why did typing `u` alone on the last line of a cell change too? The interactive window displays values with `repr`.

Step 4: `__add__`

ASK
Run `u + v` before writing anything and read the error. Who is Python asking to do the +?

EXPECTED (FOR YOU)
`TypeError: unsupported operand type(s) for +: 'Vector2D' and 'Vector2D'`. Python asks the left object, `u`, through a method with a special name.

IF STUCK, IN THIS ORDER
1. "You already wrote two methods with double underscores. What would the one for + be called?"
2. Have the learner check the guess in the Python docs on numeric types.

TASK
`Vector2D(3, 4) + Vector2D(1, 2)` gives `Vector2D(4, 6)`. After `w = u + v`, `print(u)` still shows `Vector2D(3, 4)`.

WATCH FOR
- `u` changed after `w = u + v`. The method updated `self.x` and `self.y` and returned `self`. Ask: "Does `3 + 1` change the 3?" Addition must build a new vector.
- `assert u + v == Vector2D(4, 6)` fails even when the code is right. Without `__eq__`, `==` only checks whether both sides are the same object. If time allows, take the detour, because Python asks the object here too, through `__eq__`. Otherwise compare `.x` and `.y`.

Step 5: `__mul__` by a number

TASK
`u * 2` gives `Vector2D(6, 8)` and leaves `u` unchanged.

ASK
Predict `u * 0.5`.

EXPECTED (FOR YOU)
`Vector2D(1.5, 2.0)`. An int times a float gives a float.

Step 6: the puzzle, `2 * u`

ASK
`u * 2` works. Predict `2 * u`.

EXPECTED (FOR YOU)
`TypeError: unsupported operand type(s) for *: 'int' and 'Vector2D'`

IF STUCK, IN THIS ORDER
1. "`u * 2` runs `u.__mul__(2)`. What does `2 * u` run?"
2. "Run `(2).__mul__(u)` and look at what comes back." It returns `NotImplemented`, the int's way of saying it does not know how.
3. "When the left object gives up, Python gives the right object a chance. What could that method be called?" If needed, add that the r stands for right.
4. Send the learner to `__rmul__` in the Python docs to confirm.

EXPECTED (FOR YOU)

`__rmul__`, returning the same thing as `__mul__`. Then ask why + never had this problem. Both sides were vectors, so the left one always knew how.

TASK

Stretch. Predict `sum([u, v])`. It fails with `unsupported operand type(s) for +: 'int' and 'Vector2D'`, because `sum` starts from `0` and its first step is `0 + u`. Passing a start value, `sum([u, v], Vector2D(0, 0))`, fixes it. Lesson 2's homework meets the same problem with `3 + Value(2.0)`.

Block 3: wrap-up and commit

ASK

When you write `a * b + c` with your own objects, which methods does Python call, and in what order?

EXPECTED (FOR YOU)

`a.__mul__(b)` runs first and returns a new object. Then that object's `__add__` runs with `c`. Plant the idea without explaining why it matters. In lesson 2, a `Value` class uses exactly this to record every operation as it happens.

The learner commits straight to `main`, with a message that says what they learned:

```
git add 01_python_warmup/vectors.py
git commit -m "vectors: classes, __repr__, and why 2 * u needs __rmul__"
git push
```

Then the learner writes a few sentences under "Journal" in `README.md`, covering what surprised them and what still feels fuzzy, and commits that too.

Keep the two written answers from block 1. Lesson 2 needs them.

Appendix: a reference solution

Show this only after the lesson, next to the learner's own file, and go through the differences together. It covers what the lesson asks for and nothing more.

```
# %% The class
import math


class Vector2D:
    """A vector in the plane, with coordinates x and y."""

    def __init__(self, x, y):
        """Called by Vector2D(3, 4): stores the coordinates on the new object."""
        self.x = x
        self.y = y

    def norm(self):
        """Length of the vector, by Pythagoras."""
        return math.sqrt(self.x**2 + self.y**2)

    def __repr__(self):
        """What print shows. It looks like the code that rebuilds the vector."""
        return f"Vector2D({self.x}, {self.y})"

    def __add__(self, other):
        """u + v builds a new vector and leaves u and v unchanged."""
        return Vector2D(self.x + other.x, self.y + other.y)

    def __mul__(self, k):
        """u * k, where k is a number."""
        return Vector2D(self.x * k, self.y * k)

    def __rmul__(self, k):
        """k * u. Python calls it after int.__mul__ returns NotImplemented."""
        return self * k


# %% Checks
u = Vector2D(3, 4)
v = Vector2D(1, 2)
print(u)         # Vector2D(3, 4)
print(u.norm())  # 5.0
print(u + v)     # Vector2D(4, 6)
print(u)         # Vector2D(3, 4), unchanged by the addition
print(u * 2)     # Vector2D(6, 8)
print(2 * u)     # Vector2D(6, 8)
```

Points to compare

ASK

Run `print([u, (3, 4)])`. Can you tell the vector from the tuple?

EXPECTED (FOR YOU)

A `__repr__` that returns `(3 , 4)` or `(3, 4)` prints like a tuple, so the list shows `[(3 , 4), (3, 4)]`. Python's convention is that `repr` looks like the code that rebuilds the object, which gives `[Vector2D(3, 4), (3, 4)]`. An f-string also builds the text more simply than `"(" + str(self.x) + ...`.

ASK

__rmul__ can copy the body of __mul__ or call `self * k`. If __mul__ gets a bug fix next week, which version still works?

EXPECTED (FOR YOU)

`self * k` hands the work back to __mul__, so one fix covers both `u * 2` and `2 * u`. A copied body has to be fixed twice, and forgetting one copy makes `u * 2` and `2 * u` disagree.

ASK

The checks run every time the file runs. How would you re-run only the `2 * u` check?

EXPECTED (FOR YOU)

Split the file into `# %%` cells, with the class in the top cell and the checks below, as in the reference. The project uses this format for every exercise.